

ProofArrive

by Malambi

Complete API Endpoint Flow Guide

Full integration reference for the web dashboard and mobile app.

Covers the normal trip lifecycle and all exception handling flows:

Breakdown, Accident, Goods Transfer, Overdue, and Escalation.

API Base: `/api/v1`

Auth: **Bearer JWT**

v2.0 — Exception Handling

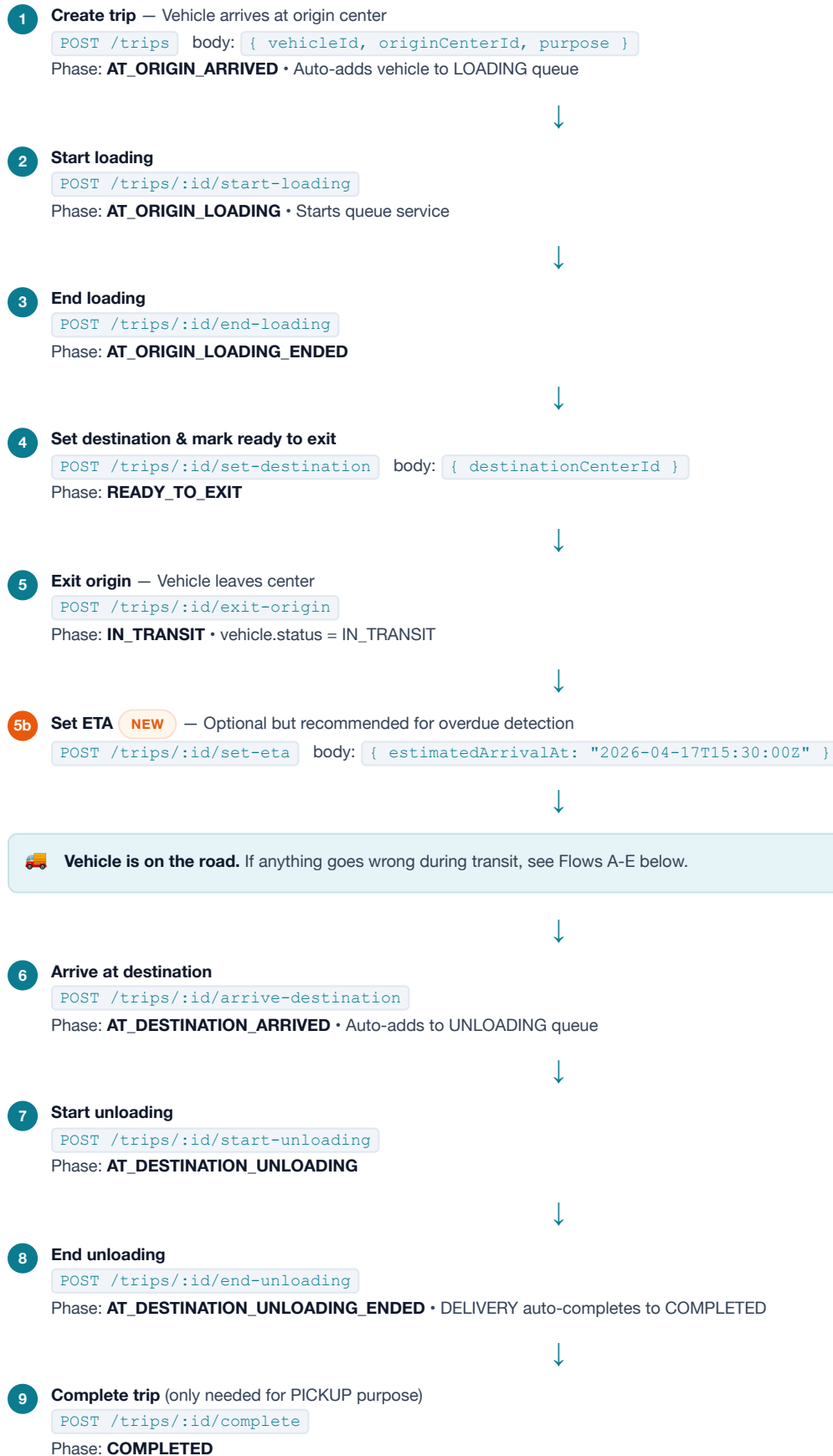
April 2026

Table of Contents

1. Normal Trip Lifecycle (Happy Path)	3
2. Flow A — Vehicle Breakdown	4
3. Flow B — Road Accident	5
4. Flow C — Goods Transfer to Rescue Vehicle	6
5. Flow D — Overdue / No-Show	7
6. Flow E — Police Stop / Other	8
7. Dashboard Page Integration Map	8
8. Mobile App Integration Map	10
9. Overdue Detection (Frontend Logic)	11
10. Complete Endpoint Reference Table	11

1 Normal Trip Lifecycle (Happy Path)

This is the standard flow when no exceptions occur. Both mobile and dashboard use the same endpoints.



READ ENDPOINTS USED THROUGHOUT

ENDPOINT	PURPOSE
<code>GET /trips/:id?include=vehicle,originCenter,destinationCenter,events</code>	Trip detail page with all relations
<code>GET /trips?status=ONGOING&include=vehicle</code>	Trips list page
<code>GET /trips/pending</code>	Trips started before today, still ongoing

2 Flow A — Vehicle Breakdown

Driver calls dispatcher. Dispatcher opens the in-transit trip on dashboard or app.

🔧 STEP 1 — REPORT BREAKDOWN (MOBILE OR DASHBOARD)

`POST /exceptions/trips/:tripId/report`

Body:

type	<code>"BREAKDOWN"</code> (required)
location	<code>"N4 highway, km 142, after Edea junction"</code> (required, free text)
description	<code>"Engine overheated. Coolant light on 20min."</code> (required, free text)

Result: trip.phase → **BREAKDOWN** • exception.status = ACTIVE • Returns exception with ID



Dispatcher chooses one of three resolution paths:

OPTION A — SEND TECHNICIAN

1. `POST /exceptions/:exceptionId/dispatch-technician`

technicianName	<code>"Jean-Paul Mbarga"</code>
technicianPhone	<code>"+237 677 123 456"</code>
estimatedArrivalTime	<code>"14:30"</code>
notes	<code>"Coming from Douala workshop"</code> (optional)

Result: trip.phase → **AWAITING_REPAIR** • exception.status = IN_PROGRESS

2. Add updates while waiting (optional, repeatable):

`POST /exceptions/:exceptionId/notes` body: `{ note: "Technician on site now" }`

3. `POST /exceptions/:exceptionId/mark-repaired`

repairedBy	<code>"Jean-Paul Mbarga"</code>
repairDescription	<code>"Radiator hose replaced. Coolant refilled."</code>
repairReference	<code>"REP-2026-042"</code> (optional)

Result: trip.phase → **IN_TRANSIT** (resumed!) • exception.status = RESOLVED_RESUMED

OPTION B — TRANSFER GOODS TO RESCUE VEHICLE

See **Flow C** (Section 4) for the full transfer flow.

OPTION C — RETURN GOODS TO ORIGIN

`POST /exceptions/:exceptionId/return-to-origin`

Body: `{ reason: "Cannot repair. No rescue vehicle available." }`

Result: trip.phase → **CLOSED_RETURNED** • trip.status = COMPLETED • exception.status = CLOSED_RETURNED

3 Flow B — Road Accident

Driver calls dispatcher, or reports directly from the mobile app (3-step form).

STEP 1 — REPORT ACCIDENT (MOBILE OR DASHBOARD)

POST /exceptions/trips/:tripId/report

FIELD	VALUE	REQUIRED
type	"ACCIDENT"	Yes
location	"N3 road, km 67, between Edea and Douala"	Yes
description	"Rear-ended by truck at roundabout"	Yes
severity	"MINOR" "MAJOR" "CRITICAL"	Yes (accident)
hasInjuries	false	No
isCargoDamaged	true	No
cargoDamageDescription	"3 pallets shifted, outer packaging torn"	No
isVehicleDriveable	false	No
policeReportReference	"PV-2026-04-001"	No

Result: trip.phase → **ACCIDENT** • exception.incidentReference = "INC-20260417-001" (auto-generated)



Same 3 resolution options as Breakdown: dispatch-technician / dispatch-rescue-vehicle / return-to-origin

 **Photos:** Uploaded separately via your file storage service. Photo URLs are stored in the `exception_photos` table. The `GET /exceptions/:id` endpoint returns the photos array.

4 Flow C — Goods Transfer to Rescue Vehicle

Triggered from a Breakdown or Accident exception when goods need to move to another vehicle.

STEP 1 — DISPATCHER SELECTS RESCUE VEHICLE (DASHBOARD)

First, list available vehicles: `GET /vehicles/by-status/AVAILABLE`

Then dispatch: `POST /exceptions/:exceptionId/dispatch-rescue-vehicle`

rescueVehicleId	<code>17592</code> (required)
transferLocation	<code>"N4 km 142, breakdown site"</code> (required)
estimatedArrival	<code>"15:00"</code> (optional)
notes	<code>"Dispatched from Douala center"</code> (optional)

Result: trip.phase → `TRANSFER_IN_PROGRESS` • exception.status = IN_PROGRESS



STEP 2 — RESCUE VEHICLE ARRIVES, STAFF SCANS QR (MOBILE)

Mobile app scans rescue vehicle QR code:

`GET /vehicles/qr-code/:qrCode` → Confirms plate matches, shows original trip info



STEP 3 — CONFIRM TRANSFER COMPLETE (MOBILE)

`POST /exceptions/:exceptionId/confirm-transfer`

cargoCountTransferred	<code>12</code> (pre-filled from manifest, editable)
cargoCondition	<code>"GOOD"</code> <code>"MINOR_DAMAGE"</code> <code>"MAJOR_DAMAGE"</code>
notes	<code>"All 12 pallets transferred. No visible damage."</code> (optional)

What happens:

- Original trip → `CLOSED_TRANSFERRED` (sealed, read-only)
- Rescue trip → auto-created, phase = IN_TRANSIT, `isRescueTrip = true`, `originalTripId = original trip ID`
- Original vehicle → status = AVAILABLE
- Rescue vehicle → status = IN_TRANSIT

Response: `{ exception, rescueTrip }`



STEP 4 — RESCUE VEHICLE ARRIVES AT DESTINATION (MOBILE — NORMAL FLOW)

The rescue trip follows the standard arrival flow:

`POST /trips/:rescueTripId/arrive-destination`

`POST /trips/:rescueTripId/start-unloading`

`POST /trips/:rescueTripId/end-unloading` → Rescue trip COMPLETED

5 Flow D — Overdue / No-Show

Triggered by frontend timer (checks ETA) or manually by dispatcher.

STEP 1 — FLAG AS OVERDUE

POST /exceptions/trips/:tripId/report

type "OVERDUE"

location "Unknown - last known: departed from Douala"

description "Vehicle 45min past expected arrival time"

Result: trip.phase → **OVERDUE** • exception.status = ACTIVE



STEP 2 — DISPATCHER INVESTIGATES

Get driver phone: GET /trips/:id?include=vehicle

Log each call attempt (repeatable):

POST /exceptions/:exceptionId/log-call-attempt

outcome "ANSWERED" | "NO_ANSWER" | "VOICEMAIL"

notes "Driver says stuck in traffic. New ETA: 16:00" (optional)

If driver gives new ETA:

POST /trips/:tripId/set-eta body: { estimatedArrivalAt: "2026-04-17T16:00:00Z" }

If driver reports breakdown/accident:

POST /exceptions/trips/:tripId/report with type: "BREAKDOWN" or "ACCIDENT" (opens Flow A or B)



STEP 3 — ESCALATE IF UNREACHABLE (AFTER 2H OR DISPATCHER'S DISCRETION)

POST /exceptions/:exceptionId/escalate

reason "DRIVER_UNREACHABLE" | "SUSPECTED_BREAKDOWN" | "SUSPECTED_ACCIDENT" | "UNKNOWN"

contactAttemptsMade 5

notes "Phone off since 14:00. Police notified." (required)

actionsTaken ["REPORTED_TO_POLICE", "NOTIFIED_MANAGEMENT"] (optional)

policeReportReference "PV-2026-04-003" (optional)

Result: trip.phase → **NO_SHOW_ESCALATED** • trip.status = COMPLETED (sealed) • exception.status = ESCALATED

6 Flow E — Police Stop / Other

Simple flow. Logs the event, dispatcher is notified, trip stays IN_TRANSIT.

REPORT FROM MOBILE

`POST /exceptions/trips/:tripId/report`

type `"POLICE_STOP"` or `"OTHER"`

location `"Checkpoint before Yaounde bypass"`

description `"Routine document check. Estimated 30min delay."`

Result: trip.phase stays `IN_TRANSIT` (no phase change) • exception.status = ACTIVE • Dispatcher sees it in active exceptions list

7 Dashboard Page Integration Map

Which API endpoint powers each UI element on the web dashboard.

7.1 Main Dashboard Page

UI ELEMENT	ENDPOINT	DATA USED
Exception Alert Banner (top)	<code>GET /exceptions/active</code>	Shows one pill per active exception: type badge + vehicle plate. "View all exceptions" links to /incidents.
KPI Card: Exceptions	<code>GET /exceptions/summary</code>	<code>totalActive</code> . Red when > 0, green "All clear" when 0.
KPI Cards: Trips, Completed, etc.	<code>GET /reports/dashboard</code>	<code>trips.ongoing</code> , <code>trips.completedInPeriod</code> , <code>exceptions.totalActive</code>
Active Exceptions Table	<code>GET /exceptions/active</code>	Type, vehicle plate, route, reported time, reported by, location, action needed. "Manage →" button.
All Active Trips Table	<code>GET /trips?status=ONGOING&include=vehicle</code>	Exception trips show colored dot + badge from trip.phase. Rescue trips have <code>isRescueTrip=true</code> .

7.2 Trips Page

FILTER CHIP	QUERY
All / Ongoing	<code>GET /trips?status=ONGOING</code>
Completed	<code>GET /trips?status=COMPLETED</code>
Exceptions (all types)	<code>GET /trips?phase=BREAKDOWN</code> + <code>ACCIDENT</code> + <code>OVERDUE</code> + <code>TRANSFER_IN_PROGRESS</code>
Breakdown	<code>GET /trips?phase=BREAKDOWN</code>
Accident	<code>GET /trips?phase=ACCIDENT</code>
Overdue	<code>GET /trips?phase=OVERDUE</code>
Transfer	<code>GET /trips?phase=TRANSFER_IN_PROGRESS</code>

7.3 Trip Detail Page

UI ELEMENT	ENDPOINT
Trip data + timeline	<code>GET /trips/:id?include=vehicle,originCenter,destinationCenter,events</code>
Exception banner + side panel	<code>GET /exceptions/trip/:tripId</code>
Exception detail (full)	<code>GET /exceptions/:id</code>
Exception timeline	<code>GET /exceptions/:id/timeline</code>
"Report exception" button	<code>POST /exceptions/trips/:tripId/report</code>
Dispatch technician	<code>POST /exceptions/:id/dispatch-technician</code>
Mark repaired	<code>POST /exceptions/:id/mark-repaired</code>
Dispatch rescue vehicle	<code>POST /exceptions/:id/dispatch-rescue-vehicle</code>
Confirm transfer	<code>POST /exceptions/:id/confirm-transfer</code>
Return to origin	<code>POST /exceptions/:id/return-to-origin</code>
Log call attempt	<code>POST /exceptions/:id/log-call-attempt</code>
Escalate	<code>POST /exceptions/:id/escalate</code>
Add note	<code>POST /exceptions/:id/notes</code>
Linked trips (transfer)	<code>GET /trips/:rescueTripId</code> — linked via <code>originalTripId</code> / <code>isRescueTrip</code>

7.4 Incidents & Exceptions Log Page [/incidents](#)

UI ELEMENT	ENDPOINT
KPI Cards (Breakdowns, Accidents, Transfers, Resolved Today)	<code>GET /exceptions/summary</code>
Incident List	<code>GET /exceptions?type=BREAKDOWN&status=ACTIVE&page=1</code>
Filter: All	<code>GET /exceptions</code>
Filter: Active	<code>GET /exceptions?status=ACTIVE</code>
Filter: Resolved	<code>GET /exceptions?status=RESOLVED_RESUMED</code>
Filter: Breakdown	<code>GET /exceptions?type=BREAKDOWN</code>
Filter: Accident	<code>GET /exceptions?type=ACCIDENT</code>
Filter: Overdue	<code>GET /exceptions?type=OVERDUE</code>
Search	<code>GET /exceptions?search=Edea</code>

8 Mobile App Integration Map

8.1 Trip Detail Screen

ACTION	ENDPOINT	CONDITION
Load trip	<code>GET /trips/:id?include=vehicle,events</code>	Always
Show "Report an issue" button	—	When trip.phase = <code>IN_TRANSIT</code>
Report Breakdown	<code>POST /exceptions/trips/:tripId/report</code> { type: "BREAKDOWN" }	User selects "Vehicle breakdown"
Report Accident	<code>POST /exceptions/trips/:tripId/report</code> { type: "ACCIDENT" }	User selects "Accident / collision"
Report Police Stop	<code>POST /exceptions/trips/:tripId/report</code> { type: "POLICE_STOP" }	User selects "Police stop / checkpoint"
Report Other	<code>POST /exceptions/trips/:tripId/report</code> { type: "OTHER" }	User selects "Other issue"

8.2 Transfer Flow (at breakdown site)

STEP	SCREEN	ENDPOINT
1	Awaiting rescue vehicle (shows plate + ETA)	<code>GET /exceptions/:id</code> → <code>rescueVehicleId</code>
2	Scan rescue vehicle QR code	<code>GET /vehicles/qr-code/:qrCode</code>
3	Confirm transfer (cargo count + condition)	<code>POST /exceptions/:id/confirm-transfer</code>
4	Success: "Trip closed. Rescue trip created."	Response includes <code>rescueTrip</code>

8.3 Incoming Arrivals (destination center)

DISPLAY	ENDPOINT / LOGIC
List incoming trips	<code>GET /trips?status=ONGOING&destinationCenterId=myCenterId&include=vehicle</code>
Normal trip	Blue dot + ETA from <code>trip.estimatedArrivalAt</code>
Overdue trip	Amber pulsing dot when <code>trip.phase = "OVERDUE"</code>
Rescue trip	Purple badge when <code>trip.isRescueTrip = true</code> . Get original: <code>GET /trips/:originalTripId</code>
QR scan arrival	<code>POST /trips/:id/arrive-destination</code> → normal unloading flow

9 Overdue Detection (Frontend Logic)

The backend provides the data. The **frontend** compares current time against the ETA.

🕒 **No GPS.** Overdue is purely time-based. The frontend polls `GET /trips?status=ONGOING&phase=IN_TRANSIT` and compares `trip.estimatedArrivalAt` against the current time.

THRESHOLD	FRONTEND ACTION	API CALL
ETA + 30 min	Show amber "Overdue" badge on trip	Optional: auto-report via <code>POST /exceptions/trips/:id/report { type: "OVERDUE" }</code>
ETA + 1 hour	Show orange alert + "Call driver" prompt	—
ETA + 2 hours	Show red alert + "Escalate" prompt	—

The dispatcher can also manually flag any trip as overdue from the trip action menu at any time.

10 Complete Endpoint Reference

Existing Endpoints (unchanged)

METHOD	ENDPOINT	USED BY	PURPOSE
POST	<code>/auth/login</code>	Both	Login with Malambi credentials
POST	<code>/auth/refresh-token</code>	Both	Refresh JWT tokens
GET	<code>/auth/profile</code>	Both	Current user profile
POST	<code>/trips</code>	Both	Create a new trip
GET	<code>/trips</code>	Both	List trips with filters
GET	<code>/trips/pending</code>	Dashboard	Ongoing trips from before today
GET	<code>/trips/:id</code>	Both	Trip detail with relations
POST	<code>/trips/:id/start-loading</code>	Mobile	Begin loading at origin
POST	<code>/trips/:id/end-loading</code>	Mobile	Finish loading at origin
POST	<code>/trips/:id/set-destination</code>	Both	Set destination + mark ready to exit
POST	<code>/trips/:id/exit-origin</code>	Mobile	Record vehicle exit from origin
POST	<code>/trips/:id/arrive-destination</code>	Mobile	Record arrival at destination
POST	<code>/trips/:id/start-unloading</code>	Mobile	Begin unloading at destination
POST	<code>/trips/:id/end-unloading</code>	Mobile	Finish unloading
POST	<code>/trips/:id/complete</code>	Both	Complete trip (PICKUP only)
GET	<code>/vehicles/by-status/:status</code>	Dashboard	List vehicles by status
GET	<code>/vehicles/qr-code/:qrCode</code>	Mobile	Validate QR code
GET	<code>/reports/dashboard</code>	Dashboard	Dashboard summary KPIs

New Endpoints NEW

METHOD	ENDPOINT	USED BY	PURPOSE
POST	<code>/trips/:id/set-eta</code>	Both	Set/update estimated arrival time
POST	<code>/exceptions/trips/:tripId/report</code>	Both	Report an exception on an in-transit trip
POST	<code>/exceptions/:id/dispatch-technician</code>	Dashboard	Dispatch technician to breakdown/accident site
POST	<code>/exceptions/:id/mark-repaired</code>	Dashboard	Mark repaired, resume trip to IN_TRANSIT
POST	<code>/exceptions/:id/dispatch-rescue-vehicle</code>	Dashboard	Dispatch available vehicle for goods transfer
POST	<code>/exceptions/:id/confirm-transfer</code>	Mobile	Confirm goods transfer, seal original trip, create rescue trip
POST	<code>/exceptions/:id/return-to-origin</code>	Dashboard	Return goods to origin, close trip
POST	<code>/exceptions/:id/log-call-attempt</code>	Dashboard	Log driver contact attempt (overdue flow)
POST	<code>/exceptions/:id/escalate</code>	Dashboard	Escalate to no-show, close trip
POST	<code>/exceptions/:id/notes</code>	Dashboard	Add dispatcher note to exception timeline
GET	<code>/exceptions</code>	Dashboard	Paginated list with filters (Incidents page)
GET	<code>/exceptions/active</code>	Dashboard	Active exceptions (dashboard banner + table)
GET	<code>/exceptions/summary</code>	Dashboard	KPI counts (breakdowns, accidents, transfers, resolved today)
GET	<code>/exceptions/trip/:tripId</code>	Both	All exceptions for a trip (trip detail side panel)
GET	<code>/exceptions/:id</code>	Both	Full exception detail with relations
GET	<code>/exceptions/:id/timeline</code>	Both	Ordered exception timeline events